

CSSE3113 – Introduction to Software Engineering

Software Quality

What is Quality?

- **Transcendental View** argues that quality is something that you immediately recognize, but cannot explicitly define.
- **User View** sees quality in terms of an end user's specific goals. If a product meets those goals, it exhibits quality.
- **Manufacturer's View** defines quality in terms of the original specification of the product. If the product conforms to the spec, it exhibits quality.
- **Product View** suggests that quality can be tied to inherent characteristics (e.g., functions and features) of a product.
- **Value-based View** measures quality based on how much a customer is willing to pay for a product.

Quality of Design Vs. Quality of Conformance

- **Quality of Design** refers to the characteristics that designers specify for a product.
- The grade of materials, tolerances, and performance specifications all contribute to the quality of design.
- It encompasses the degree to which the design meets the functions and features specified in the requirements model.
- **Quality of conformance** focuses on the degree to which the implementation follows the design and the resulting system meets its requirements and performance goals.

Software Quality

- An **effective software process** applied in a manner that creates a **useful product** that provides measurable value for those who produce it and those who use it.
 - An effective software process establishes the infrastructure that supports any effort at building a high-quality software product.
 - A useful product delivers the content, functions, and features that the end user desires, but as important, it delivers these assets in a reliable, error-free way.
 - By adding value for both the producer and user of a software product, high quality software provides benefits for the software organization and the end user community

Software Quality Dilemma

1. Good Enough Software

- Delivers high-quality functions and features that end users desire, but at the same time it delivers other more obscure or specialized functions and features that contain known as bugs.
- The software vendor hopes that the vast majority of end users will overlook the bugs because they are so happy with other application functionality
- When you deliver a good enough (buggy) product, you risk permanent damage to your company's reputation.

Software Quality Dilemma

2. Cost of Quality

- The cost of quality includes all costs incurred in the pursuit of quality or in performing quality-related activities and the downstream costs of lack of quality.
- An organization must collect metrics
 - To provide a baseline for the current cost of quality
 - Identify opportunities for reducing these costs
 - Provide a normalized basis of comparison
- The cost of quality can be divided into costs associated :
 1. Prevention
 2. Appraisal
 3. Failure
 1. Internal Failure
 2. External Failure

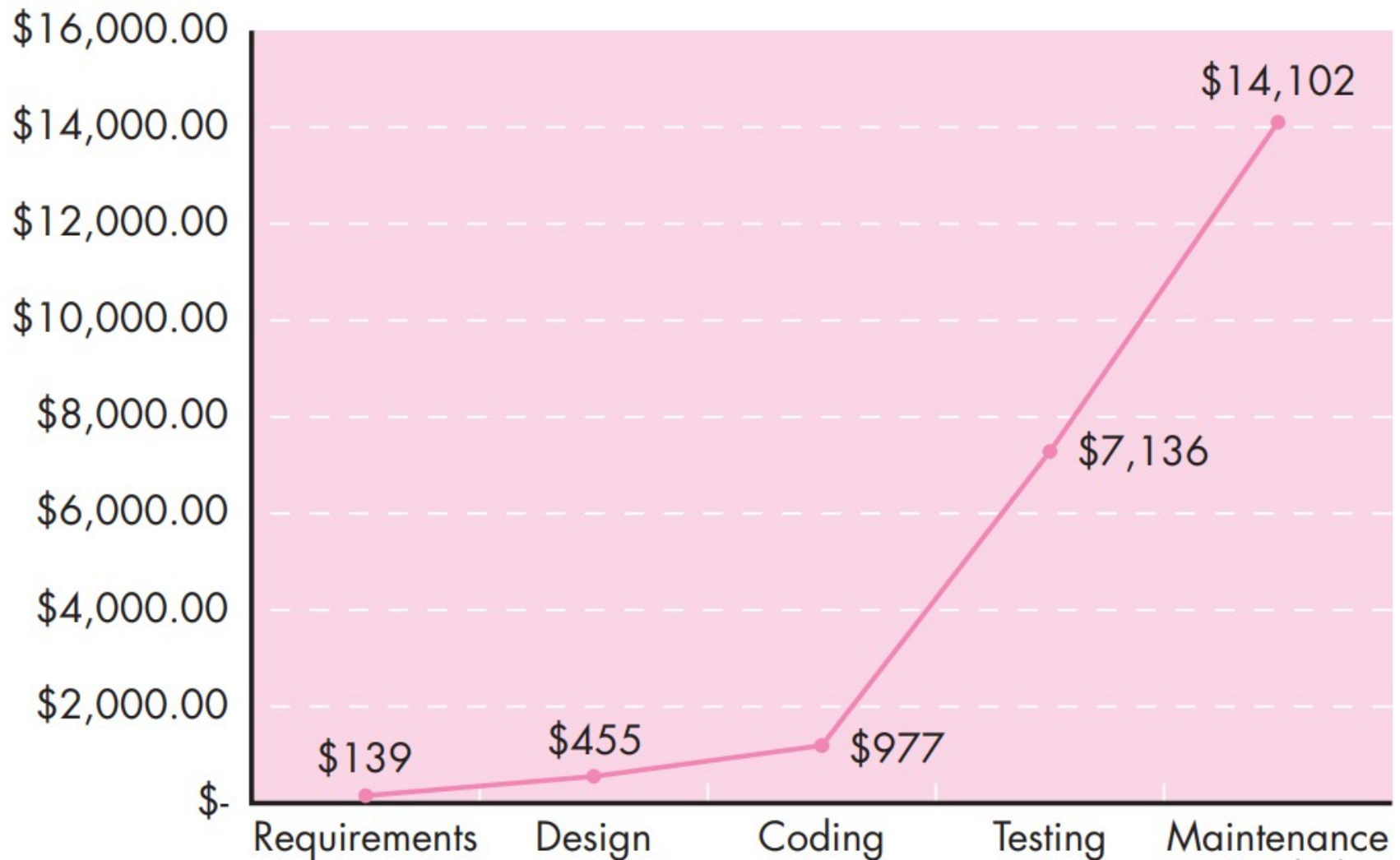
Software Quality Dilemma

- **Prevention Cost:**
 - Cost of management activities
 - The cost of added technical activities to develop complete requirements and design models
 - Test planning costs
 - The cost of all training associated with these activities
- **Appraisal costs:**
 - It include activities to gain insight into product condition the “first time through” each process.
- Examples Include:
 - The cost of conducting technical reviews for software engineering work products.
 - The cost of data collection and metrics evaluation.
 - The cost of testing and debugging

Software Quality Dilemma

- **Failure costs**
- Failure costs are those that would disappear if no errors appeared before shipping a product to customers.
- It is divided into two categories.
 1. Internal Failure Cost
 2. External Failure Cost
- **Internal failure costs** are incurred when you detect an error in a product prior to shipment.
- **External failure costs** are associated with defects found after the product has been shipped to the customer.

Relative Cost of Correcting Errors and Defects



Software Quality Dilemma

3. Risks

- Low-quality software increases risks for both the developer and the end user.

4. Negligence and Liability

- The following issues come under negligence and liability.
 - The system is late
 - The system fails to deliver desired features and functions
 - The system is error-prone
 - The system does not meet with customer approval

Software Quality Dilemma

5. Quality and Security

- As the criticality of the software system grows, application security has become increasingly important.
- Software that does not exhibit high quality is easier to hack
- To build a secure system, you must focus on quality during design.
- Elimination of architectural flaws will make it difficult to hack the software.

Software Quality Dilemma

6. The Impact of Management Actions

- Best software engineering practices can be undermined by poor business decisions and questionable project management actions.
- As each project task is initiated, a project leader will make decisions related to the following aspects that can have a significant impact on product quality.
 - Estimation Decisions
 - Scheduling Decisions
 - Risk-oriented Decisions

Achieving Software Quality

1. Software Engineering Methods

- Following activities will increase the likelihood of creating high-quality software
 - Complete understanding of the problem
 - A comprehensive design that establishes a solid foundation for the construction activity
 - Appropriate analysis and design methods

Achieving Software Quality

2. Project Management Techniques

- Use of estimation to verify that delivery dates are achievable
- Schedule dependencies are understood
- The team resists the temptation to use short cuts
- Risk planning is conducted so problems do not breed chaos
- Explicit techniques for quality and change management

Achieving Software Quality

3. Quality Control

- A set of software engineering actions that help to ensure that each work product meets its quality goals.
- Models are reviewed
- Code may be inspected in order to uncover and correct errors
- A series of testing steps is applied to uncover errors
- Measurement and feedback allows a software team to tune the process

Achieving Software Quality

4. Quality Assurance

- It establishes the infrastructure that supports software engineering methods, rational project management, and quality control.
- It consists of a set of auditing and reporting functions.
- The goal of quality assurance is to provide management and technical staff with the data necessary to be informed about product quality

Architecture Evaluation and Refinement

Cost-Benefit Analysis

- Design is iterative: we propose design decisions, assess, make adjustments, and propose more decisions
- Many techniques to evaluate the design
 - Trade-off analysis
 - Cost-benefit analysis
 - Prototyping
- A cost–benefit analysis is a widely used business tool for estimating and comparing the costs and benefits of a proposed change

Architecture Evaluation and Refinement

Cost-Benefit Analysis

- A cost-benefit analysis contrasts financial benefits with financial costs, both in Dollars
 - Costs
 - Development
 - Operational
 - Benefits
 - Reduced Operational Costs
 - Increased Earnings
- Payback period
 - the length of time before accumulative benefits recover the costs of implementation

Practice Example: Cost–Benefit Analysis

Scenario:

A software house is developing **Online Attendance System** for a university. Find annual benefit , initial cost and yearly cost.

Type	Amount (Rs.)
Development Cost	1,200,000
Hardware & Servers	400,000
Maintenance (1 Year)	200,000

Benefit Source	Amount (Rs.)
Paper cost savings	300,000
Staff workload reduction	450,000
Faster reporting & management savings	350,000

Total Initial Cost = Rs. 1,800,000

Total Annual Benefit = Rs. 1,100,000

Formula:

$\text{ROI} = (\text{Benefits} - \text{Cost}) / \text{Cost} \times 100$

$\text{Payback Period} = \text{Initial Investment} / \text{Annual Cash Inflow}$

Calculate:

ROI after 1 year

ROI after 5 years

Payback Period in years

%gain of ROI (1st year)

Total Cost (1 Year)

- $\text{ROI} = (\text{Benefits} - \text{Cost}) / \text{Cost} \times 100$
- ROI
$$= (1,100,000 - 1,800,000) / 1,800,000 \times 100$$
$$= -700,000 / 1,800,000 \times 100$$
$$= \mathbf{-38.89 \%}$$

%gain of ROI (5th year)

- **Total Cost (5 Years)**

Total Cost

= Development + Hardware + 5-Year Maintenance

= 1,200,000 + 400,000 + 1,000,000

= **Rs. 2,600,000**

- **Total Benefit (5 Years)**

Total Benefit

= Annual Benefit × 5

= 1,100,000 × 5

= **Rs. 5,500,000**

- **ROI Formula:**

$ROI = (Benefits - Cost) / Cost \times 100$

ROI

= $(5,500,000 - 2,600,000) / 2,600,000 \times 100$

= $2,900,000 / 2,600,000 \times 100$

= **111.54 %**

Project is highly profitable

Payback period

- Payback Period = Initial Investment / Annual Cash Inflow
- Initial Investment = 2,600,000
Annual Cash Inflow = 1,100,000
- Payback Period
= $2,600,000 / 1,100,000$
= **2.36 years (approx 2 years & 4 months)**

Cost Benefit Analysis of Design Protocols

TABLE 5.4 Cost–Benefit Analysis of Design Proposals

	Eliminate Noise Words	Store Indices in Bins	Add Second Server
Benefits			
Search time	0.015 sec	0.002 sec	0.008 sec
Throughput	72 requests/sec	500 requests/sec	115 requests/sec
Added value			
Costs			
Hardware			\$5,000
Software	\$50,000	\$300,000	\$200,000
Business losses	\$28,000+/yr		
Total costs first year	\$78,000	\$300,000	\$205,000

Cost Benefit Analysis of Design Protocols

For simplicity, suppose that every *additional* request per second that the system can process, up to 200 requests/second, would save the company \$2000 per year, based on retained customers and reduced calls to technical support. Given this value function, eliminating noise words would save the company \$24,000 per year, calculated as

$$(72 \text{ requests/second} - 60 \text{ requests/second}) \times \$2000/\text{year} = \$24,000/\text{year}$$

Adding a second server would save the company \$110,000 per year, calculated as

$$(115 \text{ requests/second} - 60 \text{ requests/second}) \times \$2000/\text{year} = \$110,000/\text{year}$$

The second design option would improve the system's throughput beyond what will be needed (the system will receive at most 200 requests per second). Therefore, the value added by changing to bin-based indexing is the maximum possible value:

$$(200 \text{ requests/second} - 60 \text{ requests/second}) \times \$2000/\text{year} = \$280,000/\text{year}$$

If there are multiple attributes to consider (e.g., the time it takes to update, reindex, and re-sort the catalogue), then a design's total financial added value is the sum of the added values ascribed to each of the attributes—some of whose added values may be negative if a design improves one attribute at the expense of a conflicting attribute.

Architecture Evaluation and Refinement

Cost-Benefit Analysis

- Return on Investment (ROI)
 - $ROI = \text{Benefits} / \text{Cost}$, $ROI > 1$ is desired
 - %age gain $ROI = (\text{Benefits} - \text{Cost}) / \text{Cost} \times 100$
 - Example
 - After five years
 - Cost = \$ 2249559
 - Benefits = \$ 6122893
 - $ROI = 2.72$ i.e. Total benefits are approx. 3 times the total costs
 - %age gain $ROI = 172.19\%$ i.e. Your earnings are 172.19 % of the total costs

Architecture Evaluation and Refinement

Cost-Benefit Analysis

Return on Investment (ROI) Table

Benefits/Cost, ROI > 1 is desired

	1 year	5 year	10 year
Eliminate Noise Words	$24000/78000 = 0.3076$	$(24000*5)/\{50000+(28000*5)\} = 0.6316$	$(24000*10)/\{50000+(28000*10)\} = 0.7273$
Store Indices in Bins			
Add Second Server			

Architecture Evaluation and Refinement

Cost-Benefit Analysis

- Return on Investment (ROI) Table

	1 year	5 year	10 year
Eliminate Noise Words	$24000/78000 = 0.3076$	$(24000*5)/\{50000+(28000*5)\} = 0.6316$	$(24000*10)/\{50000+(28000*10)\} = 0.7273$
Store Indices in Bins	$280000/300000 = 0.93$	$(280000*5)/300000 = 4.667$	$(280000*10)/300000 = 9.33$
Add Second Server	$110000/205000 = 0.53$	$(110000*5)/205000 = 2.683$	$(110000*10)/205000 = 5.366$

Architecture Evaluation and Refinement

Cost-Benefit Analysis

$$\% \text{age gain ROI} = (\text{Benefits} - \text{Cost}) / \text{Cost} \times 100$$

	1 year	5 year	10 year
Eliminate Noise Words	$\{(24000-78000) / 78000\} * 100$ $=$ -69.23		
Store Indices in Bins	$\{(280000-300000) / 300000\} *$ 100 $= -6.667$		
Add Second Server	$\{(110000-205000) / 205000\} * 100 =$ -46.34		

Architecture Evaluation and Refinement

Cost-Benefit Analysis

Another useful measure is the **payback period**: the length of time before accumulated benefits recover the costs of implementation. In our example, the payback period for restructuring the sorted-index module (design 2) is

$$\$300,000 / \$280,000 = 1.07 \text{ of a year} = \text{approximately 13 months}$$

Payback Period = Initial Investment / Annual Cash Inflow (or Savings)

		Years
Eliminate Noise Words	$78000 / 24000 =$	3.25 {it is wrong because 28000 is also added annually}
Store Indices in Bins	$300000 / 280000 =$	1.07
Add Second Server	$205000 / 110000 = 0.53$	1.864

Activity for Class

Comparison of Three Design Proposals

A software organization is considering **three alternative design proposals** for developing a new software system. The management wants to select the **best proposal based on Cost–Benefit Analysis and Return on Investment (ROI)** over a period of **5 years**.

Proposal	Development Cost (Rs.)	Annual Maintenance (Rs.)	Annual Benefit (Rs.)
Proposal A	1,000,000	100,000	500,000
Proposal B	1,600,000	150,000	850,000
Proposal C	2,400,000	250,000	1,200,000

Activity for Class

Proposal	Total Cost (Rs.)	Total Benefit (Rs.)	ROI (%)
A	1,500,000	2,500,000	66.67 %
B	2,350,000	4,250,000	80.85 %
C	3,650,000	6,000,000	64.38 %